

Higher Order Functions

CSC345: Programming Languages & Paradigms

Introduction

- Haskell is a functional P/L
 - A function is considered a “chunk of data” just like any other chunk of data (e.g. `Ints`)
- **Functions can be stored in variables!**
- **Functions can be passed to other functions just like any other data!**

Terminology: Functions are “*first class*”

- Higher-order functions allow common programming patterns to be encapsulated as functions.

Motivation

`doubleAll :: [Int] -> [Int]`

List Comprehension:

`doubleAll xs = [2 * x | x <- xs]`

Recursion:

`doubleAll [] = []`

`doubleAll (x:xs) = 2*x : doubleAll xs`

`tripleAll :: [Int] -> [Int]`

List Comprehension:

`tripleAll xs = [3 * x | x <- xs]`

Recursion:

`tripleAll [] = []`

`tripleAll (x:xs) = 3*x : tripleAll xs`

- The only change is the transformation, but we currently still need to write a new definition every time.
 - *Another example:* a fn that converts the `Int` into its ASCII `Char`.

Solution: the higher-order `map` function

A function is higher-order if,

- It takes a function as an argument
 - It returns a function as a result
 - Or both
-
- Question: how to generalize `doubleAll` and `tripleAll` into a higher-order function?

How to generalize `doubleAll` and `tripleAll` into a higher-order function?

Naming a function:

```
double :: Int -> Int   triple :: Int -> Int  
double x = x * 2       triple x = x * 3
```

- Creating a `mapInteger` function, recursively, using the same recursive pattern as `doubleAll` and `tripleAll`
- Will pass the function to apply as an argument (the parameter `f`)

```
mapInteger _ [] = []  
mapInteger f (x:xs) = f x : mapInteger f xs
```

Prompt: what is the type signature of `mapInteger`?

```
doubleAll [] = []  
doubleAll (x:xs) = 2*x : doubleAll xs
```

```
mapInteger _ [] = []  
mapInteger f (x:xs) = f x : mapInteger f xs
```

Prompt: what is the type signature of `mapInteger`?

```
mapInteger :: (Int -> Int) -> [Int] -> [Int]
```

- Takes a **function** and a **list of Ints**, and returns a **list of Ints**
 - Function has the type signature `(Int -> Int)`

```
mapInteger :: (Int -> Int) -> [Int] -> [Int]
mapInteger _ [] = []
mapInteger f (x:xs) = f x : mapInteger f xs
```

Prompt: Write `mapInteger` using list comprehension.

Naming a function:

```
double :: Int -> Int    triple :: Int -> Int  
double x = x * 2        triple x = x * 3
```

Using `mapInteger` and applying the functions `double` and `triple`:

```
> mapInteger double [3,5]  
[6,10]
```

```
> mapInteger triple [3,5]  
[9,15]
```

Or, defining an identifier to be the following function call:

```
sixten :: [Int]  
sixten = mapInteger double [3,5]
```


map

- **map** is a higher-order function, already defined for us in the Haskell Prelude
 - The function argument is more general than our motivating **mapInteger** example
 - Not restricted to the type **(Int -> Int)**
- Same function definitions:

Recursive:

```
map _ [] = []  
map f (x:xs) = f x : map f xs
```

List Comprehension:

```
map f xs = [f x | x <- xs]
```

- **Prompt:** what is the most general type signature?
map :: (a -> b) -> [a] -> [b]

More Examples

Another higher-order function: **filter**

- Also defined in the Haskell Standard Prelude
- **filter** selects all elements from a list that satisfy a *predicate*
 - (a predicate is a function that returns a Boolean value)

Function Signature:

```
filter :: (a -> Bool) -> [a] -> [a]
```

Recursive Function Definition:

```
filter p [] = []  
filter p (x:xs)  
  | p x      = x : filter p xs  
  | otherwise =   filter p xs
```

List Comprehension Function Definition:

```
filter p xs = [x | x <- xs, p x]
```

Examples

Using `map` and `filter` together

An example:

```
sumdoubleeven :: [Int] -> Int
```

```
sumdoubleeven xs = sum (map double (filter isEven xs))
```

Another higher-order function: `zipWith`

Combining `zip` and `map`:

- *Motivation:* what if we want to do something to corresponding elements from two lists other than making them a pair?

```
zip :: [a] -> [b] -> [(a,b)]  
  
zip (x:xs) (y:ys) = (x,y) : zip xs ys  
zip _ _ = []
```

```
zipWith f (x:xs) (y:ys) = f x y : zipWith f xs ys  
zipWith f _ _ = []
```

Function signature: `zipWith :: (a -> b -> c) -> [a] -> [b] -> [c]`

Prompt: what is a good example that demonstrates this type signature?

Other higher-order functions

These other functions are also defined in the Haskell Standard Prelude:

Decide if all elements of a list satisfy a predicate:

```
> all even [2,4,6,8]
```

```
True
```

Decide if any elements of a list satisfy a predicate:

```
> any odd [2,4,6,8]
```

```
False
```

Select elements from a list while they satisfy a predicate:

```
> takeWhile even [2,4,6,7,8]
```

```
[2,4,6]
```

Remove elements from a list while they satisfy a predicate:

```
> dropWhile even [2,4,6,7,8]
```

```
[7,8]
```